嵌入式 C 语言 · 学习路线与练习

> 配套前面的《项目深度自测题库》使用：题库告诉你**考什么**，这份告诉你**怎么学 + 动手写什么**。
> 背景：你在面试里 C 部分答不上来，本项目主题是“把 C 从‘见过’练到‘能讲、能写’”。
> 主线教材 = **你自己的 STM32 项目代码**，不是从零刷语法书。

---

0. 怎么用这份文档

1. 先看 **第 1 节目标清单**，知道面试到底考什么，别漫无目的。
2. 按 **第 2 节三步法**推进，每学一个点，用 **第 3 节**回到自己项目里找例子。
3. **第 5 节的 6 个练习必须亲手写**，写完对照参考答案；写不出来就回看第 4 节。
4. 最后用 **第 6 节**自测，三条都过，C 这关就稳了。

---

1. 目标清单：嵌入式面试真正考的

“精通 C”不是背语法，而是能**解释 + 手写**下面这些：

- **指针**：指针与数组、多级指针、函数指针、指针运算、`&arr` vs `arr`
- **内存**：栈 / 堆 / 全局(.data/.bss) / 常量 / 代码段；嵌入式为什么少用堆
- **位操作**：置位 / 清位 / 取反 / 读位 / 移位，寄存器操作
- **结构体**：对齐与填充、`#pragma pack`、位域、联合体
- **宏与预处理**：`#define` 的坑、条件编译、头文件保护
- **类型转换**：有符号/无符号、整数提升、溢出、隐式转换
- **限定符**：`const` / `static` / `volatile` 的真实含义
- **嵌入式专项**：`volatile` 修饰寄存器与 ISR 共享、中断安全、可重入

---

2. 三步学法

第一步：基础内化（约 1 周） 用“**看一段 → 关掉 → 自己写一遍**”的方式过基础：指针、数组、字符串、结构体、位操作。每学一个点，**立刻回项目里找它的影子**（见第 3 节）。

第二步：嵌入式 C 专项（约 1 周） 这是和普通 C 拉开差距的地方：`volatile`、寄存器位操作、内存映射、中断里能做什么、`static` 让变量“活下来”的机制。面试官最爱问，而你项目里到处是。

第三步：动手写小东西（持续） 光看没用，必须写。写完再对照你项目里现成的实现。练习见第 5 节。

---

3. 用你自己的项目当教材

C 主题	你项目里的现成例子
函数指针	`bl_flash.c` 跳转 APP：`void (*reset)(void) = ...; reset();`
位操作	`CAN1->MCR &= ~CAN_MCR_DBF;`、过滤器 `CAN_CMD_ID << 5`、错误码 `(esr >> 4) & 0x7`
结构体	`CAN_TxHeaderTypeDef`、`TIM_HandleTypeDef`、`CanFrame_t`
`static`	`static TIM_HandleTypeDef htim2;`、`static int32_t pid_err1;`、`static QueueHandle_t xCanRxQueue;`
`volatile`	HAL 句柄状态、ISR 与任务共享的标志/缓冲
大小端 / 字节拼接	发送 `data[0]=v&0xFF; data[1]=(v>>8)&0xFF;`，接收反向拼回
数组与指针	协议缓冲 `uint8_t buf[7+256+4]`、`&hdr[7]`、`buf[idx++]`
强制类型转换	`(uint16_t)(frame.data[1] | ((uint16_t)frame.data[2] << 8))`
宏	`#define MOTOR_DUTY_LIMIT (800U)`、`#define CAN_CMD_ID (0x201U)`

怎么用：读每个 `.c` / `.h` 时，把里面的 C 用法圈出来，逐个问“**为什么这么写？不这么写会怎样？**”。能答上来，才算懂。

---

4. 核心知识点精讲

4.1 指针

- **是什么**：指针是“存地址的变量”。`int *p;` 的 `p` 存的是一个 `int` 的地址。
- **指针 vs 数组**：数组名在表达式里**退化为首元素指针**；但 `sizeof(arr)` 是整个数组大小，`sizeof(p)` 是指针大小（32 位机 4 字节）；数组不可赋值。
- **`&arr` vs `arr`**：`arr` 是 `int*`，`&arr` 是 `int(*)[N]`，类型不同；`arr+1` 与 `&arr+1` 步长不同。
- **函数指针**：`void (*fp)(void);` 用于回调/跳转。你 Bootloader 跳 APP 就是函数指针。
- **常见坑**：野指针、悬空指针、越界、返回局部数组地址。
- **自测**：`a[i]`、`*(a+i)`、`*(&a[0]+i)` 是不是等价？为什么？

4.2 内存分区

- **代码段(.text)**：程序指令（一般只读，常在 Flash）

- **常量段**：字符串字面量、`const` 数据
- **.data**：已初始化的全局/静态变量
- **.bss**：未初始化（或初值为 0）的全局/静态变量（不占镜像空间，启动清零）
- **栈**：局部变量、函数调用；自动回收、有大小限制
- **堆**：`malloc/free`；**嵌入式少用**——碎片、时间不确定、失败难恢复
- **自测**：看项目 `.map` 或 `.list` 文件，找出 `.text/.data/.bss` 大概占用。

4.3 位操作（嵌入式必备）

- 置位 `x |= (1u << n)`；清位 `x &= ~(1u << n)`；取反 `x ^= (1u << n)`；读位 `(x >> n) & 1u`
- **为什么用 `1u`**：避免有符号左移/提升带来的问题。
- **项目例子**：`CAN1->MCR &= ~CAN_MCR_DBF;`（清调试冻结位）、过滤器 `ID << 5`、错误码 `(esr >> 4) & 0x7`。
- **常见坑**：移位位数 ≥ 位宽是未定义行为；宏参数不加括号会算错。

4.4 结构体 / 对齐

- **对齐与填充**：编译器为了访问效率会对齐成员，产生“空洞”，所以 `sizeof(struct)` 常大于成员之和。
- `#pragma pack(1)`：取消填充，用于**协议 / 存储**结构。
- **为什么解析通信帧不要直接强转结构体**：对齐、端序都依赖平台，按字节读最稳。
- **位域 / 联合体**：位域省空间但行为实现相关；联合体用于“同一块内存多种解释”（如 float ↔ 字节）。
- **自测**：`struct { char a; int b; }` 在 32 位机上 `sizeof` 是多少？为什么？

4.5 宏与预处理

- **宏 vs 函数**：宏在预处理展开，无类型检查、可能有副作用、会代码膨胀；函数有类型、有栈开销。
- **安全写法**：每个参数和整体都加括号；多语句用 `do{...}while(0)`。
- **头文件保护**：`#ifndef XXX_H / #define XXX_H / ... / #endif`。
- **条件编译**：`#ifdef DEBUG` 做调试开关。
- **自测**：`#define SQR(x) x*x`，`SQR(1+2)` 结果是多少？为什么错？

4.6 类型转换与整数提升

- **整数提升**：窄于 `int` 的类型参与运算会先提升为 `int`。
- **有符号/无符号混用**：比较时无符号占优，有符号负数会被当成很大的正数——**经典 bug**。
- **溢出**：有符号溢出是未定义行为；无符号按模回绕。
- **建议**：跨位宽运算显式转换，比如 `(int32_t)target - (int32_t)actual`。

- **自测**：`for (unsigned i = 10; i >= 0; i--)` 会怎样？

4.7 const / static / volatile

- **const**：只读；`const char *p`（内容不可改）、`char * const p`（指针不可改）含义不同。
- **static**：函数内 = 生命周期全程、初值一次；文件级 = 本文件私有。
- **volatile**：告诉编译器变量可能被“看不见的东西”改（中断、寄存器、其他任务），**禁止优化**。
- **重点误区**：`volatile` **不保证原子性**，也不是线程同步手段；它只是“每次都去内存读”。
- **项目例子**：外设寄存器访问、ISR 共享标志、DMA 状态。

4.8 嵌入式专项

- **ISR 里能做什么**：短、不阻塞、不 `printf`、不用非 `FromISR` 接口；只置标志/投递数据。
- **可重入函数**：不依赖可写的全局/静态状态。
- **寄存器操作**：用位操作读改写，注意“读-改-写”在中断里可能被抢（需临界区）。
- **定点 vs 浮点**：无 FPU 的 MCU 上浮点很慢，可用定点（你项目 PID 就是整数实现）。

---

5. 六个动手练习（含参考答案）

> 规则：**先自己写，再看答案**。写不出来就回看第 4 节对应小节。

练习 1：位操作工具宏

要求：用宏实现置位、清位、取反、读位。

```
#include <stdint.h>

#define BIT_SET(x, n)     ((x) |=  (1u << (n)))
#define BIT_CLEAR(x, n)   ((x) &= ~(1u << (n)))
#define BIT_TOGGLE(x, n)  ((x) ^=  (1u << (n)))
#define BIT_GET(x, n)     (((x) >> (n)) & 1u)
```

要点：

- **每个参数和整体都加括号**，否则优先级会坑你；
- 用 `1u` 而不是 `1`，避免有符号移位/提升问题；
- 若 `x` 是“表达式且有副作用”，宏会展开进去——稳妥做法是用 `static inline` 函数：

```
static inline void bit_set(volatile uint32_t *reg, uint8_t n) { *reg |= (1u << n); }
```

练习 2：安全字符串复制

要求：把 `src` 拷进容量为 `cap` 的 `dst`，**保证以 `\0` 结尾**；若被截断则返回 -1，正常返回复制长度。

```
#include <stddef.h>

int safe_copy(char *dst, size_t cap, const char *src)
{
    if (dst == NULL || src == NULL || cap == 0) {
        return -1;
    }
    size_t i = 0;
    while ((i + 1) < cap && src[i] != '\0') {
        dst[i] = src[i];
        i++;
    }
    dst[i] = '\0';
    return (src[i] == '\0') ? (int)i : -1;   /* src 还有内容 = 被截断 */
}
```

要点：**永远给 `\0` 留位置**（条件是 `i+1 < cap`）；`strcpy` 不检查长度，是缓冲区溢出的经典来源。

练习 3：环形缓冲区（单生产者单消费者）

要求：实现 `ring_put` / `ring_get`，满/空返回 0。

```c
#include <stdint.h>
#define RBUF_SIZE 64u          /* 建议取 2 的幂 */

typedef struct {
    uint8_t  buf[RBUF_SIZE];
    volatile uint16_t head;    /* 写入位置 */
    volatile uint16_t tail;    /* 读出位置 */
} rbuf_t;

/* 生产者 */
int ring_put(rbuf_t *rb, uint8_t byte)
{
    uint16_t next = (uint16_t)((rb->head + 1u) % RBUF_SIZE);
    if (next == rb->tail) return 0;      /* 满（留一个空位） */
    rb->buf[rb->head] = byte;
    rb->head = next;
    return 1;
}

/* 消费者 */
int ring_get(rbuf_t *rb, uint8_t *out)
{
    if (rb->head == rb->tail) return 0;  /* 空 */
    *out = rb->buf[rb->tail];
    rb->tail = (uint16_t)((rb->tail + 1u) % RBUF_SIZE);
    return 1;
}
```

要点：

- 这是**无锁**的单生产/单消费模型（ISR 放、任务取，正好对应你 CAN 队列）；
- `head/tail` 用 `volatile`，因为它可能被中断改；
- “留一个空位”区分满和空；`RBUF_SIZE` 取 2 的幂时可用 `& (RBUF_SIZE-1)` 代替取模，更快。

练习 4：状态机（协议帧解析）

要求：用 `switch + enum` 写一个解析“`A5 5A` 帧头 + CMD + 4 字节地址 + 2 字节长度”的状态机（对照你 `bl_protocol.c`）。

```c
#include <stdint.h>

typedef enum {
    S_HDR1, S_HDR2, S_CMD, S_ADDR, S_LEN0, S_LEN1
} state_t;

typedef struct {
    state_t  state;
    uint8_t  cmd;
    uint8_t  addr[4];
    uint16_t len;
    uint8_t  idx;
} parser_t;

void parser_feed(parser_t *p, uint8_t b)
{
    switch (p->state) {
    case S_HDR1:
        if (b == 0xA5u) p->state = S_HDR2;
        break;
    case S_HDR2:
        p->state = (b == 0x5Au) ? S_CMD : S_HDR1;   /* 帧头不对就重找 */
        break;
    case S_CMD:
        p->cmd = b; p->idx = 0; p->state = S_ADDR;
        break;
    case S_ADDR:
        p->addr[p->idx++] = b;
        if (p->idx == 4u) { p->state = S_LEN0; }
        break;
    case S_LEN0:
        p->len = b; p->state = S_LEN1;
        break;
    case S_LEN1:
        p->len |= (uint16_t)(b << 8);   /* 小端 */
        p->state = S_HDR1;              /* 后面继续收 DATA/CRC... */
        break;
    }
}
```

要点：状态机是**协议解析的标准结构**；你的 `bl_protocol.c` 里就是这么写的（用 `stage` 变量）。用 `enum` 比裸数字可读。

练习 5：CRC32（对照 `bl_crc.c`）

要求：用位运算实现 IEEE 802.3 CRC32，验证 `crc32("123456789") == 0xCBF43926`。

```
#include <stdint.h>

uint32_t crc32(const uint8_t *data, uint32_t len)
{
    uint32_t crc = 0xFFFFFFFFu;
    for (uint32_t i = 0; i < len; i++) {
        crc ^= data[i];
        for (uint32_t b = 0; b < 8u; b++) {
            crc = (crc & 1u) ? ((crc >> 1) ^ 0xEDB88320u) : (crc >> 1);
        }
    }
    return crc ^ 0xFFFFFFFFu;
}
```

要点：多项式 `0xEDB88320`（反射）、初值/末值都异或 `0xFFFFFFFF`；用标准向量验证正确性（你项目上位机用的是 Python `zlib.crc32`，两者一致）。

练习 6：PID（整数版 → 浮点版）

要求：看懂你项目的**增量式整数版**，再写一个**位置式浮点版**（带积分分离 + 限幅）。

```c
#include <math.h>

typedef struct {
    float kp, ki, kd;
    float integral;
    float prev_err;
    float out;
    float out_min, out_max;
    float sep_thresh;      /* 积分分离阈值 */
} pid_t;

float pid_update(pid_t *p, float target, float actual, float dt)
{
    float err = target - actual;

    /* 积分分离 + 积分限幅 */
    if (fabsf(err) <= p->sep_thresh) {
        p->integral += err * dt;
        if (p->integral > p->out_max) p->integral = p->out_max;
        if (p->integral < p->out_min) p->integral = p->out_min;
    }

    float deriv = (err - p->prev_err) / dt;
    p->out = p->kp * err + p->ki * p->integral + p->kd * deriv;

    /* 输出限幅 */
    if (p->out > p->out_max) p->out = p->out_max;
    if (p->out < p->out_min) p->out = p->out_min;

    p->prev_err = err;
    return p->out;
}
```

要点：

- 这是**位置式**（累加积分项），和你项目的**增量式**是两种形式——练习目的就是让你两种都能写、能讲区别；
- 对比点：位置式要**积分限幅**防饱和，增量式天然不容易饱和；
- 面试延伸：为什么你的项目 Kd=0？（电机惯量小 + 编码器噪声大，微分会放大噪声）

---

6. 怎么验证自己学会了

三条全过，C 这关就稳了：

1. 遮住答案，能把《项目深度自测题库》的 **C 模块（Q1–Q18）**全部讲出来；
2. 能给同学**逐行讲清** `app_can.c` 里的每个 C 用法；
3. 能不看资料默写：指针交换、位操作宏、环形缓冲、结构体对齐实验。

---

7. 资料与时间安排

资料

- 打基础：江协科技教程（你已在看）
- 进阶三件套（按清单挑章节，不必通读）：《C 和指针》《C 专家编程》《C 陷阱与缺陷》
- 嵌入式方向：《嵌入式 C 语言自我修养》
- 主线：**你自己的项目代码 + 《项目深度自测题库》C 模块**

时间（每天 1–2 小时，约 3 周）

周次	内容
第 1 周	指针、数组、字符串、位操作 + 练习 1、2
第 2 周	结构体/对齐、内存分区、宏、类型转换 + 练习 3、4
第 3 周	const/static/volatile、嵌入式专项 + 练习 5、6
持续	用题库 C 模块自测，对着项目代码逐行讲

附：和《项目深度自测题库》的关系

- **题库**：考什么（C / FreeRTOS / PID / CAN / Bootloader 全覆盖）
- **本文**：C 怎么学 + 动手写什么
- **项目代码**：真正的教材

三样一起用：看路线 → 写练习 → 回项目对照 → 用题库自测。